

R Cheat Sheet

MATH 213, Fall 2025

Table of Contents

Getting set up: Loading packages, loading data, viewing the data

Data wrangling: filter select, arrange, dropping NAs

Linear Modeling

[Cheatsheet from Rstudio github](#)

Getting set up: Loading packages, loading data, viewing the data

How to load packages (e.g. the tidyverse):

```
```{r}
library(tidyverse)
```
```

Note: Typing and playing library(tidyverse) will load the toolkit and allow us to manipulate and graph data in R

How to load data:

Method 1: From a website

```
```{r load-data}
county <-
read_csv("https://www.openintro.org/data/csv/county.csv")
```
```

Method 2 (from a “data” folder in the project):

```
```{r}
Dataframe_name <- read_csv("data/name_of_file.csv")
```
```

Note: remember, you can use the “tab” key to help fill in the folder and filename in this command.

How to take a “first look” at the data after it has been loaded:

Method 1: Once you have loaded data into RStudio, you can look at it by running a chunk containing its name:

```
```{r}
example
```
```

Method 2: To get a slightly different look, you can use the `glimpse` command.

```
```{r}
glimpse(example)
```
```

Method 3: Or the `head` command.

```
```{r}
head(example)
```
```

Method 4: Another way to look at your data is to click on the entry for the dataset in the Environment pane to the right.

One Numerical Variable: Summary Statistics

```
```{r}
oscars %>%
 summarize(mean_age = mean(age),
 median_age = median(age),
 sd_age = sd(age))
...
```{r}
oscars %>%
  summarize(max_age = max(age),
            min_age = min(age))
...
```

```
```{r}
oscars %>%
 summarize(q025 = quantile(age,
0.025),
 q25 = quantile(age, 0.25),
 q75 = quantile(age, 0.75),
 q975 = quantile(age, 0.975))
...
```{r}
oscars %>%
  group_by(award)%>%
  summarize(mean_age = mean(age))
...
```

Data Wrangling: filter, arrange, tables, dropping NAs

```
county%>% select(name,state,pop2017,median_hh_income) %>%
```

Arranging a select data from table

```
arrange(median_hh_income)
```

```
arrange(desc())
```

```
filter(variable == "variable name")
```

One Categorical Variable: Color/titles

```
```{r}
```

```
ggplot(data = GSS2,
 mapping = aes(x = marital_status, fill = marital_status))+
geom_bar()
```
```

```
```{r}
```

```
ggplot(data = GSS2,
 mapping = aes(x = marital_status, fill = marital_status))+
geom_bar() +
 scale_fill_viridis_d()
```
```

Adding a title: `labs(x = " ", y = " ", title = " ")`

One Categorical Variable: Graphing

```
```{r}
GSS2 %>%
 distinct(marital_status)
```
```

```
ggplot(data = oscars,
  mapping = aes(x = age,)) +
  geom_histogram(binwidth = 5)
```

```
ggplot(data = oscars,
  mapping = aes(x = age,
    y = 1)) +
  geom_dots()
```

```
ggplot(data = oscars,
  mapping = aes(x = age,
    y = 1)) +
  geom_jitter()
```

```
ggplot(data = oscars,
  mapping = aes(x = age,
    y = 1)) +
  geom_point()
```

```
ggplot(data = oscars,
  mapping = aes(x = age)) +
  geom_freqpoly(binwidth = 5)
```

To drop a variable:

```
```{r}

New data name<- dataname %>%
 filter(variable != "NA") %>%
 droplevels()
```
```

Flipping graph to horizontal: coord_flip() +

To add relative frequency:

```
```{r}

data_name %>%
 count(variable)%>%
 mutate(relative_freq = n / sum(n))
```
```

To reorder bars in condensing order:

Replace 'y = variable' and instead put:
fct_reorder(variable_name, prop)

Cherry blossom run

To drop missing variables from data set:

```
`{r}  
run17<-run17 %>%  
  drop_na(pace_sec)  
`
```

Stats by event:

```
run17 %>%  
  group_by(event) %>%  
  summarise(  
    mean_pace  = mean(pace_sec),  
    median_pace = median(pace_sec),  
    sd_pace    = sd(pace_sec),  
    q_0.025    = quantile(pace_sec,  
0.025),  
    q_0.25     = quantile(pace_sec, 0.25),  
    q_0.75     = quantile(pace_sec, 0.75),  
    q_0.975    = quantile(pace_sec, 0.975)  
  )
```


Grouping: Graphs

```
```{r}  
oscars %>%
 group_by(award)%>%
 summarize(mean_age = mean(age))
```
```

Height & weight

Will treat a numerical
variable as a categorical:

```
``{r}
```

```
...
```

```
  mutate(sex = factor(sex))
```

```
``
```

Scatterplots

```
labs(x = "whatever", y = "whatever1", title = "whatever1 versus whatever")
```

How to account for overplotting in a scatterplot using shape, size, and alpha.

```
ggplot(data = frederick_2019,  
       mapping = aes(x = building_area,  
                     y = price))+  
  geom_point( shape= 1, alpha= .4, size= 1, color= "red")
```

To add fancy colors

```
+ scale_color_gradient(low = "red", high = "blue")
```

```
scale_y_continuous(labels = label_dollar()) : this will change the numbers to dollars $
```

Linear Model (lm)

y first, x second

```
fit <- lm(formula = price ~ building_area, data = frederick_2019)
```

```
fit <- lm(formula = price ~ building_area + has_fireplace, data = frederick_2019)
```

Graphing a regression model together with the data:

```
ggplot(data = possum,  
       mapping = aes(x = total_l,  
                     y = head_l)) +  
  geom_point()+  
  geom_abline(intercept = 42.70979, slope = 0.57290)
```

Printing model summary output:

```
summary(fit)
```

Adding a third variable as a color

```
ggplot(data = frederick_2019,  
       mapping = aes(x = building_area,  
                     y = price,  
                     color = has_fireplace))+  
geom_jitter(shape = 1, size = 1, alpha = 0.5)+  
scale_y_continuous(labels = label_dollar())
```

All variables as a line:

```
ggplot(data = frederick_2019,  
       mapping = aes(x = building_area,  
                     y = price,  
                     color = has_fireplace)) +  
geom_jitter(shape = 1, size = 1, alpha = 0.5)+  
geom_line(mapping = aes(x = building_area , y = .fitted, color = has_fireplace))+  
scale_y_continuous(labels = label_dollar())
```

Linear Models: residual plots

Augmenting

```
body_dims <- fit %>%  
  augment(body_dims)
```

Residuals vs x-values

```
`{r}  
ggplot(data = body_dims,  
       mapping = aes(x = height_in,  
                     y = .resid))+  
  geom_point()+  
  geom_abline(intercept = 0, slope = 0, linetype =  
"twodash")  
`
```

Residuals vs predicted y-values

```
ggplot(data = body_dims,  
       mapping = aes(x = .fitted,  
                     y = .resid))+  
  geom_point()+  
  geom_abline(intercept = 0, slope =  
0, linetype = "twodash")
```

Residual histogram

```
ggplot(data = body_dims,  
       mapping = aes(x = .resid))+  
  geom_histogram()
```

Frederick houses (1,2,3)

To add 2 lines to the graph

```
geom_line(mapping = aes(x = building_area , y = .fitted, color = has_fireplace))
```